

Manifesto Final de Cutover Técnico — Plataforma REVISA

1. Objetivo do cutover

Este manifesto consolida a passagem da especificação técnica para uma implantação controlada do backend da Plataforma REVISA no monorepo, com foco em:

- materialização correta dos arquivos no repositório
- aplicação ordenada das migrations
- carga inicial de papéis, permissões e usuário administrativo
- validação mínima de autenticação, RBAC, escopo e persistência
- ativação do pipeline de CI com PostgreSQL real
- redução de ambiguidade operacional na entrada em desenvolvimento/homologação

Este documento é o ponto único de referência para o **primeiro cutover técnico** do backend.

2. Escopo do cutover

Este cutover cobre:

- backend FastAPI
- Alembic + PostgreSQL
- JWT + RBAC
- auditoria centralizada mínima
- routers e domínios iniciais
- testes de integração mínimos
- CI de backend com PostgreSQL real

Não cobre ainda, nesta etapa:

- frontend web completo
 - app mobile completo
 - observabilidade avançada
 - refresh token com revogação robusta
 - testes de carga
 - deploy em produção
 - secrets manager e hardening final de segurança
-

3. Critérios para iniciar o cutover

Antes de iniciar, confirmar:

- existe um repositório monorepo criado ou reservado
- a equipe decidiu a branch base de implantação inicial

- Python 3.11+ está disponível
- PostgreSQL 16 está disponível localmente ou via container
- a equipe aceitou a estrutura modular proposta
- os arquivos técnicos anteriores foram aprovados como baseline

Se algum destes itens não estiver pronto, o cutover não deve começar.

4. Ordem oficial de colagem dos arquivos

A colagem deve seguir a ordem abaixo para evitar quebra de imports, inconsistência de bootstrap e falhas de migração.

Bloco 1 — estrutura raiz do monorepo

Colar primeiro:

- `.env.example`
- `.gitignore`
- `Makefile`
- `README.md`
- `docker-compose.yml`
- `.github/workflows/api-ci.yml`

Bloco 2 — estrutura do app backend

Criar estrutura de pastas:

- `apps/api/app/core/`
- `apps/api/app/shared/`
- `apps/api/app/api/deps/`
- `apps/api/app/api/v1/routers/`
- `apps/api/app/domain/iam/`
- `apps/api/app/domain/core/`
- `apps/api/app/domain/territory/`
- `apps/api/app/domain/polo/`
- `apps/api/app/domain/workflow/`
- `apps/api/app/domain/analytics/`
- `apps/api/app/tests/integration/`
- `apps/api/alembic/versions/`
- `apps/api/scripts/`
- `database/seeds/`

Bloco 3 — arquivos de configuração do backend

Colar em seguida:

- `apps/api/pyproject.toml`
- `apps/api/Dockerfile`
- `apps/api/alembic.ini`

- apps/api/alembic/env.py
- apps/api/app/main.py

Bloco 4 — núcleo técnico do backend

Colar nesta ordem:

- apps/api/app/core/settings.py
- apps/api/app/core/database.py
- apps/api/app/core/security.py
- apps/api/app/core/auth.py
- apps/api/app/core/permissions.py
- apps/api/app/core/scope.py
- apps/api/app/core/audit.py
- apps/api/app/core/startup.py
- apps/api/app/shared/audit.py

Bloco 5 — dependências FastAPI

Colar em seguida:

- apps/api/app/api/deps/auth.py
- apps/api/app/api/deps/permissions.py
- apps/api/app/api/deps/scope.py

Bloco 6 — domínios do backend

Colar na ordem: 1. domain/iam/models.py 2. domain/iam/repository.py 3. domain/core/models.py 4. domain/core/schemas.py 5. domain/core/repository.py 6. domain/core/service.py 7. domain/territory/models.py 8. domain/territory/schemas.py 9. domain/territory/repository.py 10. domain/territory/service.py 11. domain/polo/models.py 12. domain/polo/schemas.py 13. domain/polo/repository.py 14. domain/polo/service.py 15. domain/workflow/models.py 16. domain/workflow/schemas.py 17. domain/workflow/repository.py 18. domain/workflow/service.py 19. domain/analytics/repository.py 20. domain/analytics/service.py

Bloco 7 — roteamento da API

Colar na ordem:

- apps/api/app/api/v1/routers/__init__.py
- apps/api/app/api/v1/routers/auth.py
- apps/api/app/api/v1/routers/persons.py
- apps/api/app/api/v1/routers/contacts_capture.py
- apps/api/app/api/v1/routers/polos.py
- apps/api/app/api/v1/routers/tasks.py
- apps/api/app/api/v1/routers/dashboards.py
- apps/api/app/api/v1/api.py

Bloco 8 — scripts e seeds

Colar:

- `apps/api/scripts/seed_initial_data.py`
- `apps/api/scripts/bootstrap_admin.py`
- `database/seeds/permissions_seed.sql`

Bloco 9 — migrations Alembic

Colar por ordem numérica:

- `0001_create_iam_schema.py`
- `0002_create_core_schema.py`
- `0003_create_territory_schema.py`
- `0004_create_polo_schema.py`
- `0005_create_events_schema.py`
- `0006_create_workflow_schema.py`
- `0007_create_governance_schema.py`
- `0008_create_analytics_schema.py`

Bloco 10 — testes

Colar por último:

- `apps/api/app/tests/conftest.py`
- `apps/api/app/tests/integration/test_auth_login.py`
- `apps/api/app/tests/integration/test_persons.py`
- `apps/api/app/tests/integration/test_contacts_capture.py`
- `apps/api/app/tests/integration/test_polos.py`
- `apps/api/app/tests/integration/test_tasks.py`
- `apps/api/app/tests/integration/test_dashboards.py`

5. Sequência oficial de migrations

A aplicação das migrations deve ser estritamente sequencial.

Ordem

1. `0001_create_iam_schema`
2. `0002_create_core_schema`
3. `0003_create_territory_schema`
4. `0004_create_polo_schema`
5. `0005_create_events_schema`
6. `0006_create_workflow_schema`
7. `0007_create_governance_schema`
8. `0008_create_analytics_schema`

Justificativa da ordem

- `iam` precisa existir antes de qualquer tabela referenciando usuários
- `core` precisa existir antes de entidades organizacionais e pessoas
- `territory` depende de `iam` e `core`
- `polo` depende de `territory` e `core`
- `events` depende de `polo` e `core`
- `workflow` depende de `territory`, `polo`, `core` e `iam`
- `governance` depende de `iam` e `core`
- `analytics` depende de praticamente todos os anteriores

Comando oficial

```
cd apps/api
alembic upgrade head
```

Regra obrigatória

Não editar migrations já coladas manualmente depois da primeira aplicação em ambiente compartilhado. Qualquer ajuste estrutural posterior deve virar nova migration.

6. Sequência oficial de bootstrap e seed

Depois das migrations, a carga inicial deve seguir esta ordem:

Etapa 1 — permissões

Aplicar o catálogo de permissões:

- `database/seeds/permissions_seed.sql`

Etapa 2 — roles

Aplicar roles base via `seed_initial_data.py`

Etapa 3 — role_permissions

Associar papéis e permissões canônicas via `seed_initial_data.py`

Etapa 4 — escopos padrão

Garantir que tipos de escopo previstos sejam reconhecidos no bootstrap lógico:

- `GLOBAL`
- `REVISA`
- `VEREADOR`
- `GABINETE`
- `POLO`

- EQUIPE
- SELF

Etapa 5 — admin inicial

Executar:

```
python scripts/bootstrap_admin.py
```

Resultado esperado

- permissões carregadas
- roles carregadas
- matriz role-permission carregada
- usuário admin criado

7. Checklist de implantação local

7.1 Checklist pré-subida

Marcar cada item apenas quando estiver validado:

- ☐ repositório criado com estrutura correta
- ☐ arquivos-base colados sem alterações indevidas
- ☐ `.env` criado a partir do `.env.example`
- ☐ PostgreSQL disponível e acessível
- ☐ `DATABASE_URL` válido
- ☐ `TEST_DATABASE_URL` válido
- ☐ ambiente virtual criado
- ☐ dependências instaladas com `pip install -e .`
- ☐ migrations coladas na pasta correta
- ☐ imports entre módulos sem erro de caminho

7.2 Checklist de banco

- ☐ conexão com banco validada
- ☐ `alembic upgrade head` executado com sucesso
- ☐ schemas criados: `iam`, `core`, `territory`, `polo`, `events`, `workflow`, `governance`, `analytics`
- ☐ materialized view `analytics.mv_vereador_dashboard` criada
- ☐ nenhuma migration pendente

7.3 Checklist de seed

- ☐ permissões inseridas
- ☐ roles inseridas
- ☐ role_permissions inseridas

- [] admin inicial criado
- [] hash de senha do admin gerado corretamente

7.4 Checklist de subida da API

- [] `uvicorn app.main:app --reload` sobe sem traceback
 - [] endpoint `/health` responde `200`
 - [] `/api/v1/auth/login` responde para usuário válido
 - [] `/api/v1/auth/me` responde com token válido
 - [] endpoints protegidos negam acesso sem token
 - [] endpoints protegidos negam acesso sem permissão
-

8. Comandos oficiais de implantação local

8.1 Criar ambiente

```
cp .env.example .env
cd apps/api
python -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e .
```

8.2 Aplicar migrations

```
alembic upgrade head
```

8.3 Rodar seed lógico

```
python scripts/seed_initial_data.py
```

8.4 Criar admin inicial

```
python scripts/bootstrap_admin.py
```

8.5 Subir API

```
uvicorn app.main:app --reload
```

8.6 Rodar testes

```
pytest -q
```

9. Validação pós-subida

A validação pós-subida deve ser objetiva e executável.

9.1 Validação de saúde

Executar:

- GET /health

Esperado:

- status 200
- body: {"status": "ok"}

9.2 Validação de login

Executar:

- POST /api/v1/auth/login

Payload:

```
{
  "username": "admin",
  "password": "Admin@123"
}
```

Esperado:

- status 200
- access_token
- refresh_token
- token_type = Bearer

9.3 Validação de usuário autenticado

Executar:

- GET /api/v1/auth/me
- header Authorization: Bearer <token>

Esperado:

- id do usuário
- username admin
- roles
- permissions
- scopes

9.4 Validação de proteção de rota

Executar sem token:

- GET /api/v1/persons
- GET /api/v1/tasks
- GET /api/v1/vereadores/{id}/dashboard

Esperado:

- 401 ou 403

9.5 Validação de persistência

Executar com token e permissão:

- POST /api/v1/persons

Payload mínimo:

```
{
  "full_name": "Teste de Cutover"
}
```

Esperado:

- 201
- id gerado
- registro persistido em core.persons
- log em governance.audit_logs

9.6 Validação de captação

Executar com token:

- POST /api/v1/contacts-capture

Payload mínimo:

```
{
  "origin": "MOBILE",
  "classification": "CIDADA0",
  "full_name": "Contato de Teste"
}
```

Esperado:

- 201

- registro persistido em `territory.contacts_capture`
- log de auditoria criado

9.7 Validação de tarefa

Executar com token:

- `POST /api/v1/tasks`

Payload mínimo:

```
{
  "task_type": "FOLLOW_UP",
  "title": "Tarefa de teste"
}
```

Esperado:

- `201`
- registro persistido em `workflow.tasks`
- auditoria criada

10. Pipeline de CI — ativação oficial

Depois da validação local, ativar o pipeline com PostgreSQL real.

Checklist de CI

- [] workflow `.github/workflows/api-ci.yml` colado
- [] paths corretos no trigger
- [] Python 3.11 definido
- [] serviço PostgreSQL 16 configurado
- [] variáveis de ambiente configuradas
- [] `pip install -e .` executa sem erro
- [] `alembic upgrade head` executa sem erro
- [] `pytest -q` executa sem erro

Critério de aceite do CI

O cutover só é considerado concluído quando o pipeline passar em:

- push na branch de trabalho
- pull request contra branch principal

11. Ordem oficial de validação humana

A validação humana deve seguir a ordem abaixo:

1. estrutura do repositório
2. instalação local
3. migrations
4. seed
5. admin bootstrap
6. health check
7. login
8. auth/me
9. endpoints protegidos sem token
10. criação de pessoa
11. criação de captação
12. criação de tarefa
13. pipeline CI verde

Não inverter a ordem.

12. Critérios objetivos de sucesso do cutover

O cutover técnico será considerado bem-sucedido apenas quando todos os itens abaixo forem verdadeiros:

- backend sobe localmente sem erro
 - migrations aplicam até `head`
 - seeds iniciais executam com sucesso
 - admin inicial autentica com sucesso
 - RBAC nega acesso indevido
 - criação de pessoa funciona
 - criação de captação funciona
 - criação de tarefa funciona
 - auditoria grava logs de mutação
 - testes mínimos passam
 - CI com PostgreSQL real passa
-

13. Plano de rollback

Se o cutover falhar antes da validação pós-subida:

- descartar branch de cutover ou reverter commits
- dropar base de homologação/local
- recriar banco limpo
- reaplicar apenas artefatos aprovados

Se falhar após migrations, mas antes do aceite:

- restaurar banco limpo
- revisar migration com falha
- não editar migrations já executadas em ambiente compartilhado; criar nova migration corretiva

Se falhar no CI:

- congelar merge
 - corrigir apenas arquivos do backend afetados
 - rerodar pipeline
-

14. Riscos operacionais mais prováveis

Risco 1 — imports quebrados

Mitigação:

- seguir a ordem oficial de colagem
- validar `uvicorn` logo após colagem do backend

Risco 2 — migrations com ordem incorreta

Mitigação:

- usar exatamente a sequência definida
- não renomear revision ids

Risco 3 — seed incompleto

Mitigação:

- separar permissões, roles e role_permissions
- validar contagem após seed

Risco 4 — autenticação funciona, mas RBAC não

Mitigação:

- validar `/auth/me`
- conferir roles e permissions no banco

Risco 5 — testes locais passam, CI falha

Mitigação:

- testar com PostgreSQL real localmente antes do push
 - não depender de SQLite para schemas PostgreSQL
-

15. Manifesto executivo de execução

A equipe deve tratar este cutover como uma operação de implantação controlada, e não como simples colagem de arquivos.

Princípios executivos:

- não improvisar estrutura
 - não alterar nomes de módulos durante a colagem
 - não pular migrations
 - não misturar correções estruturais com validação funcional
 - não considerar o backend “pronto” antes de CI verde
-

16. Sequência final resumida

1. Criar estrutura do monorepo
 2. Colar arquivos-base raiz
 3. Colar backend core
 4. Colar deps FastAPI
 5. Colar domínios
 6. Colar routers
 7. Colar scripts e seeds
 8. Colar migrations
 9. Instalar dependências
 10. Aplicar migrations
 11. Rodar seeds
 12. Criar admin
 13. Subir API
 14. Validar health/login/auth/me
 15. Validar create person/capture/task
 16. Rodar testes
 17. Subir branch
 18. Validar CI com PostgreSQL real
 19. Aprovar cutover
-

17. Fechamento

Este manifesto é o documento final de transição entre arquitetura e implantação inicial do backend REVISA.

Ele consolida:

- ordem de materialização
- sequência de migrations
- bootstrap
- seed

- checklist de implantação
- validação pós-subida
- ativação do CI
- rollback

Com este documento, a equipe tem um roteiro único para executar o primeiro cutover técnico com disciplina e rastreabilidade.